Assignment 1

CSA0903-Java Programming

Name: Sowmiya. D

Reg No.:192511063

Scenario — Campus Smart-Parking System

The university campus has a multi-level parking facility used by students, staff and visitors. The facility has several floors, and each floor has a fixed number of numbered slots. Vehicles entering the campus may be two-wheelers, cars or electric vehicles (EVs). EVs may also use charging slots, which are charged separately.

When a vehicle enters, the system must allocate a suitable free slot, issue a parking ticket with a unique ticket number, and record the entry time. When the vehicle leaves, the system must release the slot and compute the parking fee. Fees depend on the vehicle type and the duration of parking. Staff and students holding a valid monthly pass park free of charge, while visitors pay the full fee.

The facility may contain:

1 floor × 20 slots → 3 floors × 50 slots → 5 floors × 100 slots

You are required to analyse the problem, design an object-oriented model for it, implement the model in Java and validate it. Your design must remain correct and easy to extend as the facility and its rules grow.

Design Challenge:

The university now wants to add new vehicle categories and a new pricing rule without rewriting the classes that already work.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 6 – Design the Inheritance Hierarchy 4 Marks

Bloom's Level: K4 – Analyse

Design an inheritance hierarchy for the vehicles and, if appropriate, for the users (student, staff, visitor).

Discuss:

- The superclass and subclasses, and the type of inheritance used (single, multilevel or hierarchical)
- What is inherited and what is added in each subclass
- How super is used for fields, methods and constructors
- The order in which constructors execute when an ElectricCar object is created, shown by a trace

Draw the hierarchy as a **UML class diagram**.

Alternative Concept and Complexity Analysis

You used inheritance (Car is-a Vehicle). If composition were used instead – a single Vehicle class that has-a VehicleType object holding the rate – what would the solution be? Compare the work needed to add a new vehicle type and analyse the time to compute a fee and the space per vehicle object in both designs.

Evidence Expected

A justified inheritance hierarchy, UML diagram, use of super and a traced constructor execution order.

Solution:

TASK 6 – Design the Inheritance Hierarchy

Campus Smart-Parking System – 4 Marks

1. Inheritance Hierarchy

Inheritance allows one class to acquire the fields and methods of another class. In the Campus Smart-Parking System, inheritance is used to represent different vehicle types and user categories.

Vehicle hierarchy

- Vehicle – Superclass containing common vehicle details.
- TwoWheeler – Subclass representing two-wheelers.
- Car – Subclass representing cars.
- ElectricCar – Subclass of Car representing electric cars.

The design uses both hierarchical and multilevel inheritance.

- Hierarchical inheritance: Vehicle → TwoWheeler and Vehicle → Car.
- Multilevel inheritance: Vehicle → Car → ElectricCar.

User hierarchy

- User – Superclass containing common user details.
- Student – Subclass representing student users.
- Staff – Subclass representing college staff.
- Visitor – Subclass representing visitors.

This uses hierarchical inheritance.

2. UML Class Diagram

Campus Smart-Parking System – Inheritance UML

Conceptual class hierarchy. For the written diagram, ElectricCar extends Car; Student, Staff and Visitor extend User.

3. Inherited and Added Members

Class	Inherited members	New members
Vehicle	–	vehicleNumber, ownerName, baseRate
TwoWheeler	Vehicle fields and methods	helmetRequired
Car	Vehicle fields and methods	numberOfSeats
ElectricCar	Vehicle and Car members	batteryCapacity
User	–	userId, name
Student	User members	rollNumber
Staff	User members	employeeId
Visitor	User members	visitPurpose

4. Use of the super Keyword

The super keyword refers to the immediate parent class.

It is used in three ways:

1. Access parent fields: super.vehicleNumber
2. Call parent methods: super.displayInfo()
3. Call parent constructor: super(vehicleNumber, ownerName, baseRate)

Example:

```
class Vehicle {
    String vehicleNumber;
    String ownerName;
    double baseRate;


    Vehicle(String vehicleNumber, String ownerName, double baseRate) {
        this.vehicleNumber = vehicleNumber;
        this.ownerName = ownerName;
        this.baseRate = baseRate;
        System.out.println("Vehicle constructor");
    }
```

```
    void displayInfo() {
        System.out.println("Vehicle: " + vehicleNumber);
    }
}
class Car extends Vehicle {
    int numberOfSeats;
    Car(String vehicleNumber, String ownerName,
        double baseRate, int numberOfSeats) {
        super(vehicleNumber, ownerName, baseRate);
        this.numberOfSeats = numberOfSeats;
        System.out.println("Car constructor");
    }
}

class ElectricCar extends Car {
    double batteryCapacity;

    ElectricCar(String vehicleNumber, String ownerName,
                double baseRate, int numberOfSeats,
                double batteryCapacity) {

        super(vehicleNumber, ownerName, baseRate, numberOfSeats);
        this.batteryCapacity = batteryCapacity;
        System.out.println("ElectricCar constructor");
    }

    @Override
    void displayInfo() {
```

```
        super.displayInfo();
        System.out.println("Battery Capacity: " + batteryCapacity + " kWh");
    }
}
```

5. Constructor Execution Trace

When an ElectricCar object is created:

```
ElectricCar e1 = new ElectricCar(
    "TN01EV1234", "Arun", 20.0, 5, 75.0
);
```

The constructor execution order is:

1. Vehicle constructor

Initializes vehicle number, owner name and base rate.

2. Car constructor

Initializes the number of seats.

3. ElectricCar constructor

Initializes battery capacity.

Output

```
Vehicle constructor
Car constructor
ElectricCar constructor
```

Java executes the superclass constructor before the subclass constructor.

6. Alternative Concept – Composition vs Inheritance

Composition means a class contains an object of another class instead of inheriting from it.

For example, a Vehicle class can have a VehicleType object that stores the vehicle category and parking rate.

Comparison	**Inheritance**	**Composition**
Design	Car extends Vehicle	Vehicle has a VehicleType
Adding a new type	Create a new subclass	Create a new VehicleType
Fee calculation	Uses subclass rate or overridden method	Uses VehicleType rate

Flexibility	Less flexible when types change	More flexible
Object space	Parent and subclass fields	Vehicle fields plus a type reference
Maintenance	May create many subclasses	Type-specific logic is separated

Time Complexity

- Inheritance: Fee calculation takes O(1) time when the subclass directly provides its rate.
- Composition: Fee calculation also takes O(1) time when the VehicleType object directly provides its rate.

Space Complexity

Let k be the number of vehicle types and n be the number of vehicles.

- Inheritance: Each vehicle object stores common fields and its subclass-specific fields. Total vehicle storage is O(n), with class definitions for k types.
- Composition: Each vehicle stores a reference to a VehicleType object. If type objects are shared, the additional type storage is O(k), making the overall storage O(n + k). If every vehicle creates its own type object, additional storage can be O(n).

Conclusion: Inheritance is suitable for the Campus Smart-Parking System because vehicles share common properties but have different characteristics. Composition is more flexible when new vehicle types and parking rates need to be added frequently.

